

AI Coding Agents:
Need **Specs**, Not Vibes.

Agenda

- | | | |
|----|---|---------|
| 01 | SDD (Spec-Driven Development) | Pag. 2 |
| 02 | Windsurf | Pag. 10 |
| 03 | Starting a Greenfield Project with Spec Kit | Pag. 12 |

01 SDD (Spec-Driven Development)

The Problem Everyone Recognizes

AI can write code fast. So why do we still get wrong implementations?

- Fast output does not guarantee correct output
- Vague prompts create hidden assumptions
- The first version often looks right, but misses intent
- Rework starts after code already exists

Code generation is cheap now, but good code still has a cost, and new habits are needed around coding agents.

Hour 1	Hour 3	Hour 5
		
"This AI is amazing!"	"Why isn't this working?"	"git reset --hard"
App is 70% done in 20 minutes	Fixing one bug creates three more	Back to square one

Vibe Coding and Why It Breaks

Example prompt:

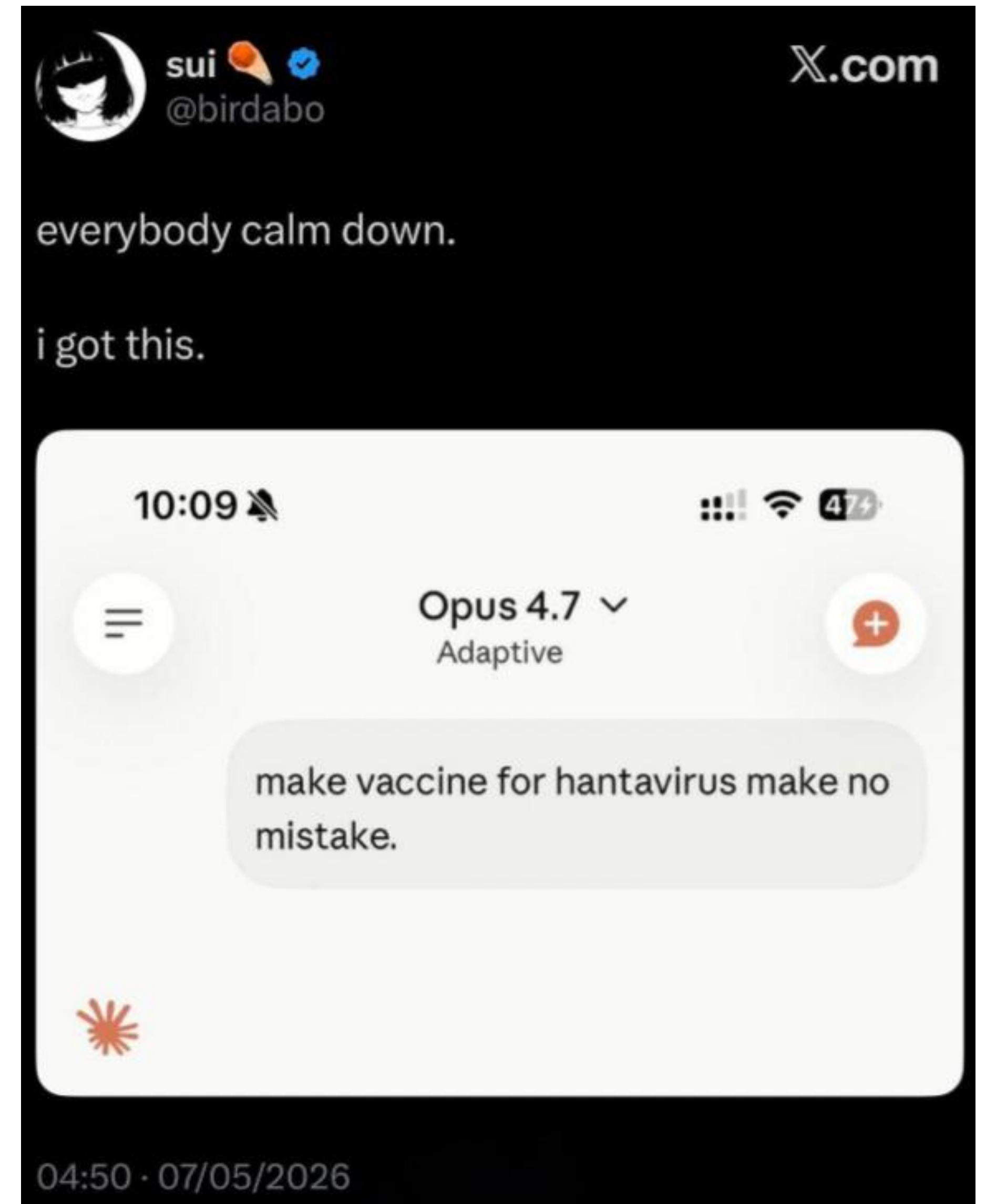
"Build me a project management app."

Now the agent must **guess**:

- User roles
- Permissions
- Workflows
- Data model
- Edge cases
- Success criteria

Vibe coding is useful for exploration. It is risky for delivery.

AI-generated code is only good as the instructions that created it!
Garbage in → Garbage out



What SDD Actually Changes

Spec-Driven Development changes the source of truth.

Instead of:

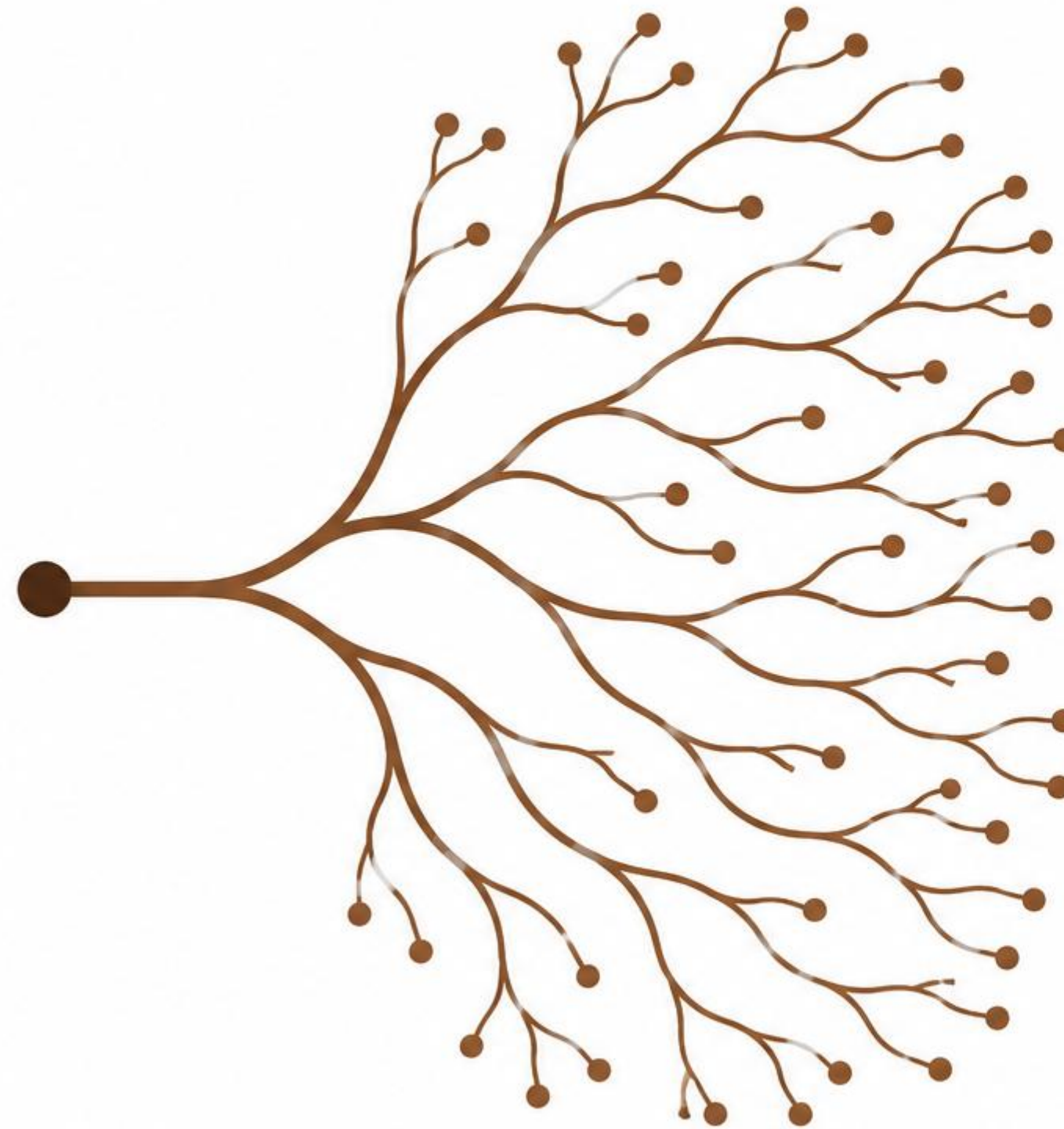
- Prompt first
- Code first
- Requirements discovered later
- Agent makes hidden decisions

SDD means:

- Intent first
- Spec first
- Code follows the spec
- Result is verified against the spec

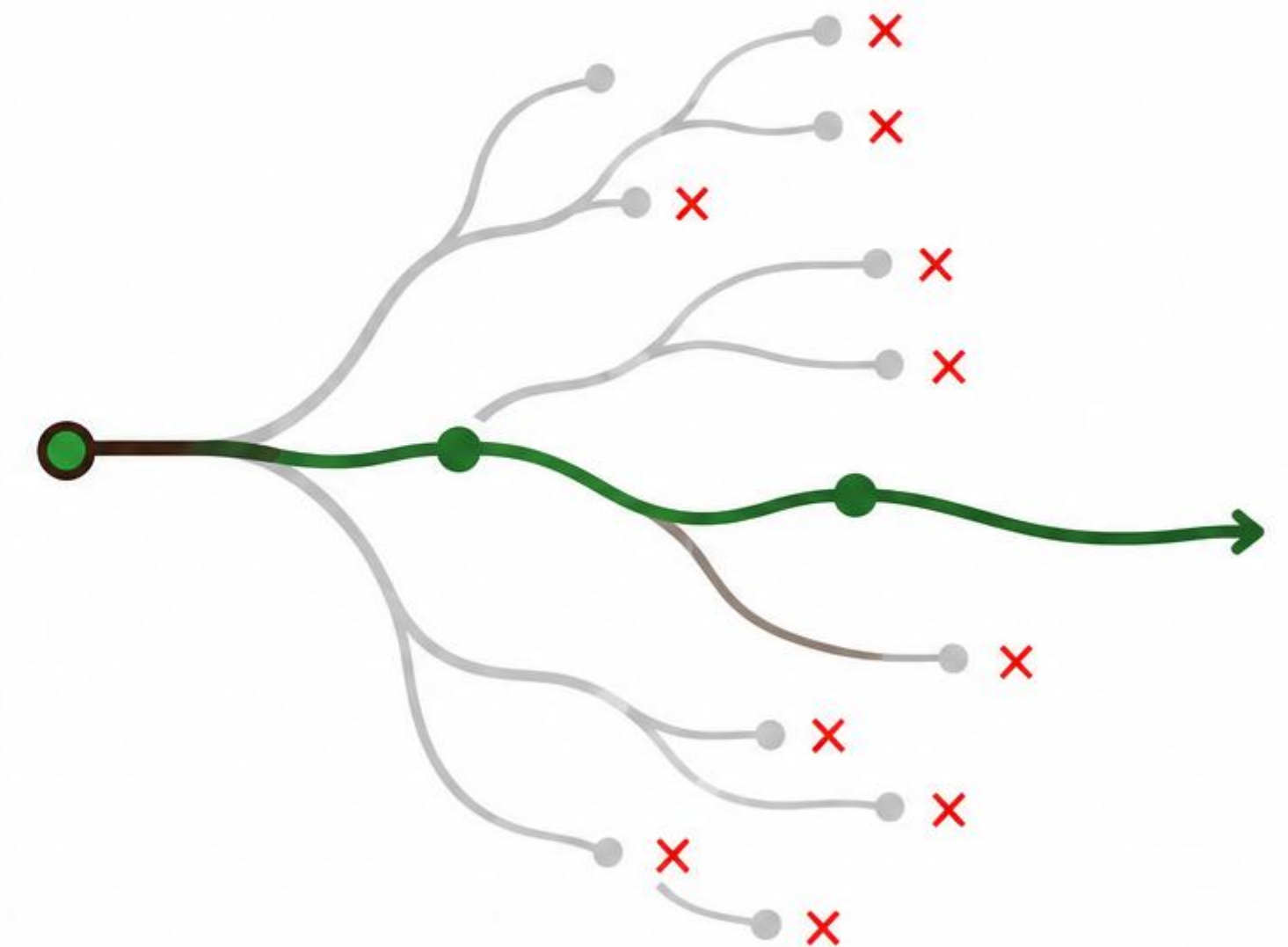
VIBE CODING PROMPT

“Build me a project management app.”



SDD PROMPT (SPECIFICATION)

“Build a project management app with detailed requirements, edge cases, constraints, and acceptance criteria.”



What a Good Spec Contains

A good spec answers:

1. What should be true when this is done?
2. What is in scope?
3. What is explicitly out of scope?
4. What constraints must be respected?
5. What decisions are already fixed?
6. How will we verify success?

Useful sections:

- Outcome
- Scope / non-goals
- Constraints and assumptions
- Key decisions
- Acceptance criteria
- Verification notes

How SDD Works in Practice

Framework-neutral SDD workflow:

1. Start with intent
2. Clarify behavior
3. Define scope and non-goals
4. Capture constraints
5. Add acceptance criteria
6. Let the agent implement
7. Review against the spec

SDD is a way to reduce ambiguity before code is written.

When to Use SDD

- The work is real, shared, or long-lived
- Acceptance criteria matter
- Multiple people or sessions are involved
- The feature affects architecture
- You are changing an existing system
- Mistakes are expensive to reverse

When Not To

- Small bug fixes / trivial tweaks
- The task is tiny
- The work is exploratory
- The result is disposable
- Mistakes are cheap

02

Windsurf

Windsurf Orientation

- **Subscription**
Cognition Platform (Enterprise)
- **Project explorer**
- **Terminal**
- **Settings**
Disable: Auto Execution, Auto Web Requests, Auto-Continue
- **Cascade**
- **Modes**
Ask, Plan, Code
- **Models**
AGPT vs Claude, Thinking, Pricing

03 Starting a Greenfield Project with Spec Kit

GitHub Spec Kit Installation Guide



Prerequisites

- Python
- uv for package management



Installation

`uvx --from git+https://github.com/github/spec-kit.git specify init --here`

Or just download the sample project with already installed spec kit from:

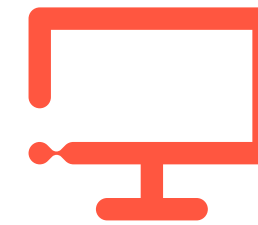
<https://github.com/MTrajK/engineering-notes/>

GitHub Spec Kit Folders Structure



.specify folder

all the Spec Kit related configuration and artefacts are located here (scripts, templates, memories)



.windsurf folder

all the windsurf configuration and artefacts are located here (workflows, skills, rules)

Constitution

`/speckit.constitution`

The constitution should not be invented by the agent. It should be distilled from project intent, delivery standards, and quality expectations.

The constitution is not the product brief. It is the project's technical operating system.

- **Runs once at project start**
- **Defines long-lived rules for the project**
- **Should include:**
 - architecture principles
 - tech stack constraints
 - code quality expectations
 - testing expectations
 - security/privacy rules
 - accessibility/logging/error handling
 - do not put detailed feature requirements here

CONSTITUTION.md

Used by GitHub Spec Kit

Project principles and governance used by Spec Kit.

AGENTS.md

Used by AI Coding Agents (Windsurf in this case)

Agent behavior and project conventions used by Windsurf/Cascade.

Use AGENTS.md if you want durable, team-shared instructions for Cascade.

It is not where the product vision belongs.

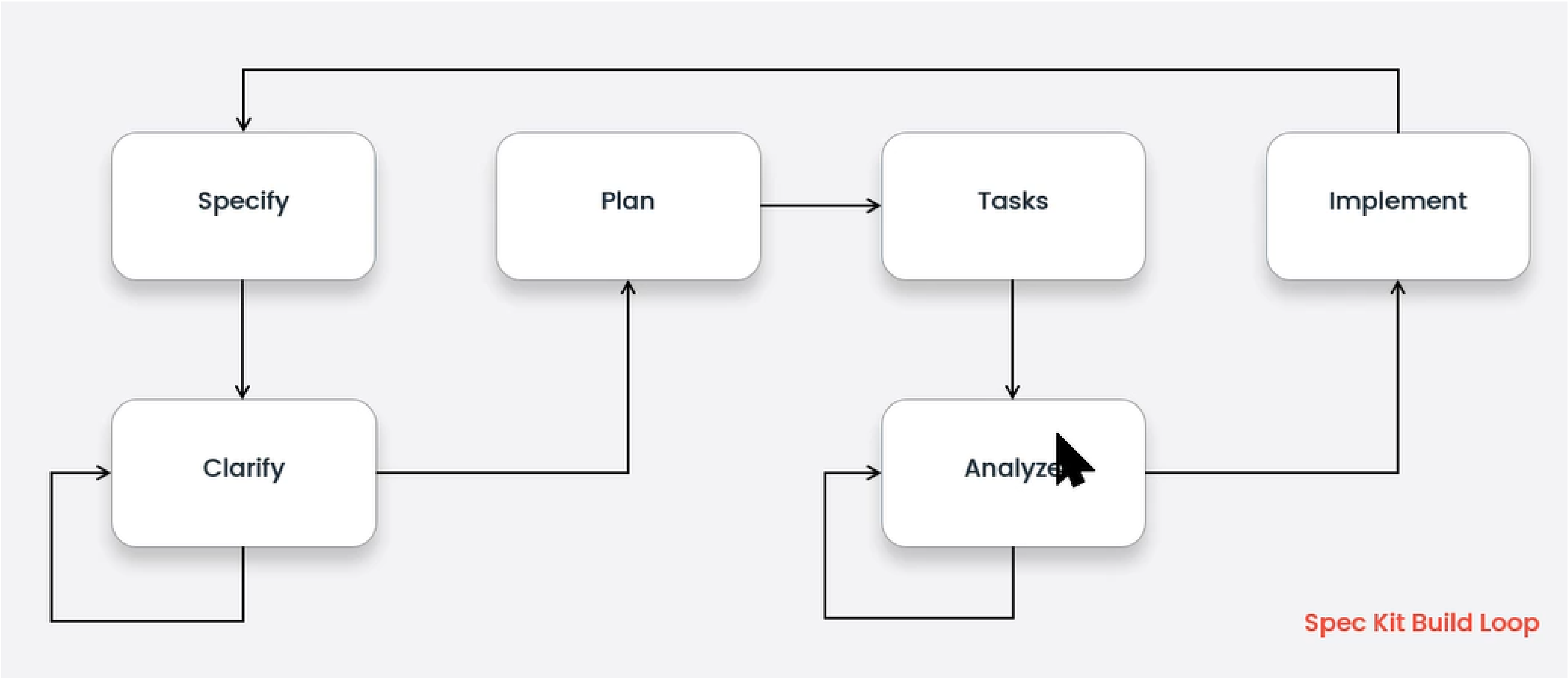
README.md

Used by humans

Human-facing project introduction.

Project's front door for humans: quick starts, project descriptions, and contribution guidelines.

GitHub Spec Kit Ceremony



Business / Product Requirements

/speckit.specify

- Describe what and why
- Focus on users, scenarios, rules, outcomes
- Avoid tech stack here

/speckit.clarify

- Resolve ambiguity before planning

Technical Implementation

/speckit.plan

- Technical approach, stack, architecture

/speckit.tasks

- Ordered implementation tasks

/speckit.analyze

- Consistency check before coding

/speckit.implement

- Agent executes the plan

Q&A

Thank *you*!